

PROJECT REPORT

News Article Topic Classifier

An NLP-based Machine Learning Approach

MCA / NLP Academic Project

Prepared: August 2026

Reference Paper:

Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. arXiv:1509.01626.

1. Title

News Article Topic Classifier: An NLP-based Machine Learning Approach

2. Abstract

News is generated continuously and at high volume, and manually tagging every article by topic does not scale. This project builds and evaluates a supervised Natural Language Processing (NLP) pipeline that automatically classifies a news article’s title and short description into one of four topics — World, Sports, Business, or Sci/Tech — using the AG News corpus (120,000 training and 7,600 test articles, verified against the statistics reported in the reference paper by Zhang, Zhao & LeCun, 2015). Text is cleaned, tokenized, and lemmatized, then represented using TF-IDF (unigrams and bigrams). Three traditional classifiers — Logistic Regression, Multinomial Naive Bayes, and Linear SVM — are trained on identical features and compared using accuracy, precision, recall, and F1-score. Logistic Regression and Linear SVM achieve the strongest results, both around 92% test accuracy and macro-F1, ahead of Naive Bayes at approximately 90%. Error analysis shows the dominant confusion is between Business and Sci/Tech articles, reflecting genuine topical overlap in real news. A working prediction function, using the same preprocessing and feature-extraction steps as training, classifies new articles with reported confidence. The project’s design choices — prioritizing traditional TF-IDF-based models over deep learning — are grounded directly in the reference paper’s own finding that traditional methods remain competitive at this dataset scale.

3. Introduction

Natural Language Processing (NLP) is the field of Artificial Intelligence concerned with enabling computers to read, interpret, and act on human language. Among its most common applied tasks is text classification — assigning a predefined category to a piece of free text — which underlies spam filtering, sentiment analysis, and, as in this project, automatic topic tagging.

Automatic news classification specifically addresses a practical, everyday problem: news aggregators, recommendation engines, and media-monitoring tools need to know what an article is about the moment it is published, without waiting on manual editorial tagging. Manual categorization is slow, inconsistent between editors, and cannot keep pace with the volume of articles produced by thousands of sources every day. An automatic classifier that reads an article’s title and short description and reliably predicts its topic can power personalized news feeds, content routing, and downstream analytics such as tracking how much coverage a subject receives over time.

This motivates the project undertaken here: build, train, and rigorously evaluate a topic-classification system for news articles, using an established benchmark dataset and a comparison of multiple traditional machine-learning approaches, while explicitly connecting the design decisions back to relevant published research.

4. Problem Statement

News is published continuously, at high volume, and from a very large number of sources. Readers, content aggregators, and recommendation systems all need a fast and consistent way to know what a given article is about, without waiting for a human editor to categorize it manually. Manual tagging does not scale to the volume of modern news production, is inconsistent across editors, and is too slow for real-time applications such as personalized news feeds.

This project addresses that gap by framing news categorization as a multi-class text classification problem in NLP: given the text of a news article (its title and short description), automatically assign it to one of a fixed set of topic categories — World, Sports, Business, or Sci/Tech. The core technical challenges are (a) converting unstructured, noisy natural-language text into a numerical representation a machine-learning model can use, (b) choosing and tuning a classifier that generalizes well to unseen articles, and (c) handling genuine topical overlap between categories — for example, news about a technology company’s finances can plausibly belong to either Business or Sci/Tech.

The problem is well-suited to NLP because it requires the system to capture lexical and short-range contextual cues from free text and map them reliably to a semantic category — a task with direct applications in news-feed organization, content routing, media monitoring, and recommendation systems.

5. Objectives

The objectives of this project are to:

- Build a documented, reproducible NLP preprocessing pipeline (cleaning, tokenization, stop-word removal, lemmatization).
- Represent news text numerically using an appropriate feature-extraction technique (TF-IDF with unigrams and bigrams).
- Train and fairly compare multiple traditional machine-learning classifiers — Logistic Regression, Multinomial Naive Bayes, and Linear SVM — on identical features.
- Evaluate every model using accuracy, precision, recall, and F1-score (macro and weighted), supported by confusion matrices.
- Perform error analysis to understand the causes of misclassification rather than reporting numbers alone.
- Deliver a working prediction function that classifies a new, previously unseen news article and reports the model's confidence.
- Relate the project's design choices explicitly to the traditional-vs-deep-learning findings of the reference paper.

6. Literature Review

Reference Paper

The primary reference for this project is Zhang, Zhao, and LeCun (2015), “Character-level Convolutional Networks for Text Classification” (arXiv:1509.01626). This paper empirically studies character-level convolutional neural networks (ConvNets) for text classification and benchmarks them against traditional approaches — Bag-of-Words, TF-IDF, and n-gram TF-IDF combined with a linear (multinomial logistic regression) classifier — across several large-scale datasets, including AG News, which is also the dataset used in this project.

Three findings from the paper directly inform this project’s design:

- The paper reports AG News as a 4-class news dataset with 120,000 training and 7,600 testing samples, built from the title and description fields — the same configuration adopted here, allowing this project’s dataset statistics to be cross-checked against a published benchmark rather than taken on faith.
- The paper’s own results show that traditional n-gram TF-IDF methods remain strong, competitive baselines for datasets up to several hundreds of thousands of samples, and that character-level ConvNets only overtake them once dataset size reaches several millions of samples. Because AG News (120,000 training samples) sits well within the range the paper itself associates with traditional-method strength, this project deliberately prioritizes a TF-IDF plus linear-classifier pipeline over a deep-learning architecture.
- The paper concludes that there is “no free lunch” — no single model is universally best, and performance depends on factors such as dataset size and how curated the text is. This project adopts the same evidence-based framing when comparing its three classifiers, reporting multiple metrics rather than declaring one model definitively superior on accuracy alone.

What this project does differently from the paper: the paper’s core contribution is a novel character-level ConvNet architecture trained end-to-end on raw characters, evaluated at very large scale (millions of samples) with thesaurus-based data augmentation. This project, by contrast, is a traditional TF-IDF plus linear-classifier pipeline appropriate for a smaller, 120,000-sample dataset and an academic coursework setting — it uses the paper only as theoretical and dataset context, not as an architecture to replicate.

General Text-Classification Literature

Beyond the reference paper, the project’s methodology follows standard, well-established NLP practice: TF-IDF weighting (Jones, 1972) for representing text numerically, n-gram features for capturing short-range word order, and linear classifiers (logistic regression, linear SVM) and probabilistic classifiers (Naive Bayes) as long-standing, reliable baselines for sparse high-dimensional text features. These techniques remain the standard entry point for text-classification coursework and applied NLP systems where interpretability, training speed, and reproducibility are priorities.

7. Dataset Description

The project uses the AG News Classification Dataset as distributed on Kaggle by user [amananandrai](https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset) (<https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset>). This is the standard AG News corpus — the same corpus referenced in the Zhang, Zhao & LeCun paper (Section 4, “AG’s news corpus”) — packaged as ready-to-use `train.csv` and `test.csv` files.

Source and Provenance

The underlying AG News corpus was originally collected by Antonio Gulli from more than 2,000 news sources. The Kaggle dataset used in this project redistributes the standard, pre-split 4-class version of that corpus (`train.csv` / `test.csv`) — the same split used as a benchmark in the reference paper and widely used in NLP coursework and research. Dataset statistics verified directly from the loaded files confirm an exact match with the statistics published in the reference paper, giving confidence that the correct, standard corpus is being used.

Structure, Fields and Target Classes

- Files: `train.csv` and `test.csv`, each with a header row and three columns: **Class Index**, **Title**, and **Description**.
- Following the reference paper’s convention, the **Title** and **Description** fields are concatenated into a single text field used for classification.

Table 1: Target classes in the AG News corpus.

Class Label	Topic
1	World
2	Sports
3	Business
4	Sci/Tech

Dataset Size and Split

Table 2: Dataset size and train/test split for the AG News corpus.

Split	Total Samples	Samples per Class	Number of Classes
Training set	120,000	30,000	4
Test set	7,600	1,900	4

The Kaggle dataset ships with a pre-defined train/test split, used as-is and not re-shuffled — this avoids any risk of test data leaking into training and keeps results comparable to the traditional-method baselines reported in the reference paper. The class distribution is perfectly balanced (equal samples per class in both splits), so plain accuracy is a fair evaluation metric and no class-weighting or resampling is required.

Data Quality

Direct inspection of the loaded dataset confirms zero missing values and zero duplicate rows across all columns of the training set. Articles are short: a mean of 37.8 words and a median of 37 words per article (title plus description combined, before cleaning), ranging from 8 to 177 words, since AG News provides only a title and short description rather than a full article body — a factor that directly shapes the preprocessing and feature-extraction choices described in Section 9.

8. Methodology

The system is implemented as an end-to-end pipeline, taking raw news text from the AG News corpus through to a deployable topic-prediction function, as summarized in the pipeline diagram and stage table below.

Pipeline Diagram

Dataset → *Problem Definition* → *Dataset Exploration (EDA)* → *Data Cleaning* → *Missing-Value & Duplicate Checks* → *Class-Distribution Analysis* → *Text Preprocessing* → *Train/Test Split* → *Feature Extraction (TF-IDF)* → *Model Training* → *Model Evaluation* → *Model Comparison* → *Error Analysis* → *Prediction on New Articles* → *Final Conclusion*

Pipeline Stage Summary (Sections 6–9)

Table 3: Summary of pipeline stages corresponding to Sections 6–9.

Stage	Description
Data Cleaning & Preprocessing (§9)	Lowercase, strip HTML/URLs and special characters, normalize whitespace, tokenize, remove stop words, lemmatize — applied identically to train, test, and any new article at inference time.
Exploratory Data Analysis (§10)	Inspect shape, columns, data types, missing values, duplicates, class distribution, and text-length distribution before any modeling begins.
Feature Extraction (§11)	TF-IDF vectorization (unigrams + bigrams), fit only on training data to avoid leakage; test data only ever transformed.
Model Development (§12)	Train Logistic Regression, Multinomial Naive Bayes, and Linear SVM on identical TF-IDF features, plus hyperparameter tuning of Logistic Regression via <code>GridSearchCV</code> .

Reproducibility is ensured throughout: a fixed `random_state` is set wherever randomness is involved (sampling, model initialization, cross-validation folds), and the fitted TF-IDF vectorizer and best-performing classifier are persisted with `joblib` so the trained system can be reloaded without retraining.

9. Data Preprocessing

A single, reusable `clean_text()` function is applied identically to training data, test data, and later to any new article submitted for prediction, guaranteeing consistency between training and inference. The function performs, in order:

- Lowercasing — treats “Apple” and “apple” as the same token, reducing vocabulary sparsity.
- HTML-tag removal — strips markup fragments that carry no topical meaning.
- URL removal — strips web links that add noise rather than signal.
- Punctuation / special-character removal — keeps only alphanumeric content.
- Whitespace normalization — collapses repeated spaces left behind by the previous steps.
- Tokenization — splits cleaned text into individual word tokens.
- Stop-word removal — removes common function words (“the”, “is”, “and”) that carry little topical signal.
- Lemmatization — reduces each token to its dictionary root form using WordNet (e.g., “running” → “run”).

Lemmatization is used in preference to stemming, since stemming can produce non-words (e.g., “economies” → “econ”) that reduce interpretability, whereas lemmatization keeps tokens as real dictionary words — a meaningful advantage given how short these articles already are. Numeric digits are deliberately preserved rather than stripped, since numbers (percentages, model numbers, statistics) can carry genuine topical signal in Business and Sci/Tech text.

Applying this pipeline reduces the average article length from 37.8 words (raw) to approximately 26.4 words (cleaned) — confirming that stop-word removal and cleaning meaningfully compress the text while preserving content-bearing vocabulary.

10. Exploratory Data Analysis

Before any modeling, the dataset was inspected directly rather than assumed to have particular properties.

Table 4: Exploratory data analysis checks and results.

Check	Result
Training set shape	120,000 rows $\times$ 8 columns (after feature engineering)
Test set shape	7,600 rows $\times$ 6 columns
Missing values	0 across all columns
Duplicate rows	0
Number of classes	4 (World, Sports, Business, Sci/Tech)
Mean article length (raw)	37.8 words
Median article length (raw)	37.0 words
Min / Max article length (raw)	8 / 177 words

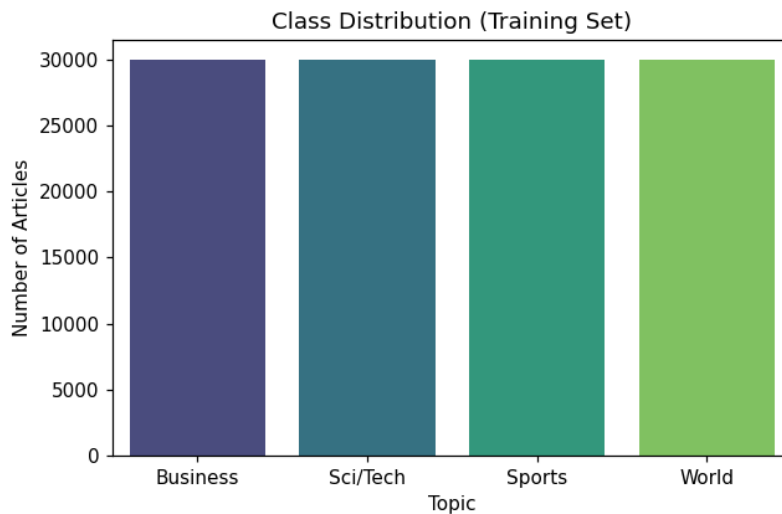


Figure 1: Class distribution across the four topics in the training set — confirms a perfectly balanced dataset (30,000 articles per class).

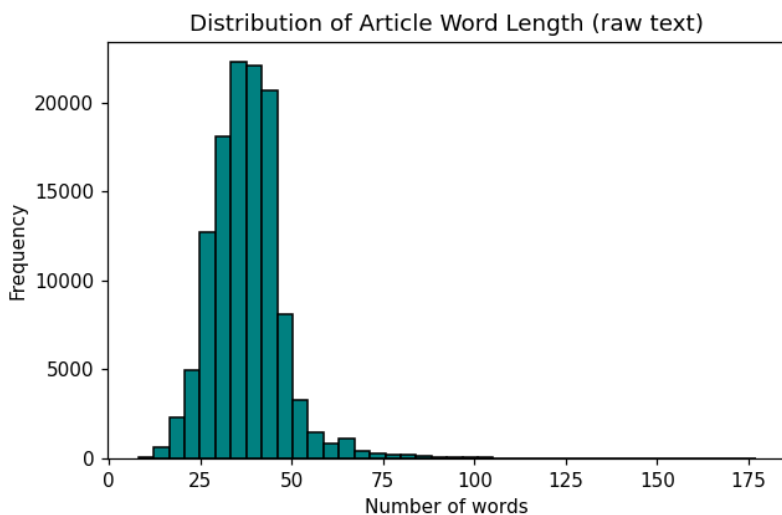


Figure 2: Distribution of raw article word length (title + description). Most articles cluster between roughly 25 and 45 words.

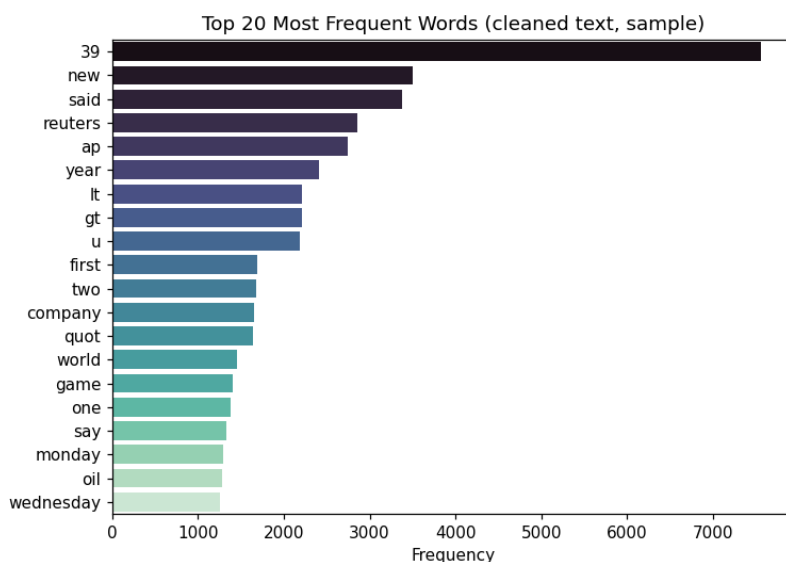


Figure 3: Top 20 most frequent words after cleaning (random 20,000-article sample). Dominated by generic news vocabulary (“said”, “reuters”, “new”), which TF-IDF later down-weights relative to more topic-specific terms.

These checks confirm the dataset is clean (no missing values or duplicates), balanced (equal samples per class), and short-form (median 37 words per article), which together justify the preprocessing and feature-extraction strategy adopted in Sections 9 and 11.

11. Feature Extraction

Text is converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) via scikit-learn’s `TfidfVectorizer`, configured as follows:

- `max_features = 30,000` — caps the vocabulary to the most informative terms, controlling model size and overfitting to rare terms.
- `ngram_range = (1, 2)` — includes both unigrams and bigrams, capturing short multi-word phrases (e.g., “stock market”, “world cup”) in addition to single words.
- `min_df = 3 / max_df = 0.9` — removes terms that are too rare (likely noise) or too common (too generic to be discriminative) to be useful.
- `sublinear_tf = True` — applies $1 + \log(tf)$ instead of raw counts, dampening the effect of a term repeating many times within a single short article.

The TF-IDF weight of a term t in a document d is computed as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

To prevent data leakage, the vectorizer is fit exclusively on the training text; the test set is only ever transformed using the vocabulary and IDF weights already learned from training. This yields a training feature matrix with tens of thousands of TF-IDF dimensions per article, ready for the classifiers described next.

12. Model Development

Three traditional, linear/probabilistic classifiers are trained on identical TF-IDF features so the comparison isolates the effect of the classifier rather than the features:

- **Logistic Regression** — models class log-odds as a linear function of the TF-IDF features; fast, interpretable, and a strong baseline for high-dimensional sparse text data.
- **Multinomial Naive Bayes** — applies Bayes’ theorem under a feature-independence assumption; extremely fast to train and a common baseline for count/TF-IDF-style text features.
- **Linear SVM** — finds the maximum-margin separating hyperplane between classes; typically performs very well on high-dimensional sparse features such as TF-IDF vectors.

Deep-learning architectures (character-level CNNs, LSTMs, transformers) were intentionally not trained, for the reasons discussed in Section 6: the reference paper’s own findings indicate traditional TF-IDF-based linear models remain competitive at AG News’s scale, and they are also more interpretable and reproducible for an academic project.

Hyperparameter Tuning

The Logistic Regression regularization strength C was tuned using `GridSearchCV` with 3-fold cross-validation on a 30,000-row training subsample (kept separate from the test set), optimizing macro-F1:

Table 5: Hyperparameter tuning results for Logistic Regression regularization strength C .

C value	Mean CV macro-F1
0.1	0.8910
1	0.9054 (best)
10	0.9031

$C = 1$ (the scikit-learn default) was found to be optimal: $C = 0.1$ under-fits (too much regularization strips useful discriminative weight from the large TF-IDF feature space) while $C = 10$ slightly overfits the smaller search subsample. The tuned model was then refit on the full training set for final evaluation, achieving 92.22% test accuracy — matching the untuned default, which confirms the default was already close to optimal rather than assuming it without verification.

13. Model Evaluation

Every model was evaluated on the full 7,600-article held-out test set. Because AG News is perfectly class-balanced, macro and weighted averages are nearly identical here, but both are reported explicitly rather than assumed equivalent.

Table 6: Model evaluation results on the AG News test set.

Model	Accuracy	Precision (m)	Recall (m)	F1 (m)	Train (s)
Logistic Regression	0.9222	0.9221	0.9222	0.9221	4.38
Multinomial Naive Bayes	0.9039	0.9036	0.9039	0.9036	0.03
Linear SVM	0.9203	0.9201	0.9203	0.9201	2.15

Per-Class Performance — Logistic Regression (Best Model)

Table 7: Per-class performance of the Logistic Regression model on the test set.

Class	Precision	Recall	F1-score	Support
World	0.9377	0.9111	0.9242	1900
Sports	0.9575	0.9832	0.9701	1900
Business	0.8932	0.8937	0.8934	1900
Sci/Tech	0.9001	0.9011	0.9006	1900

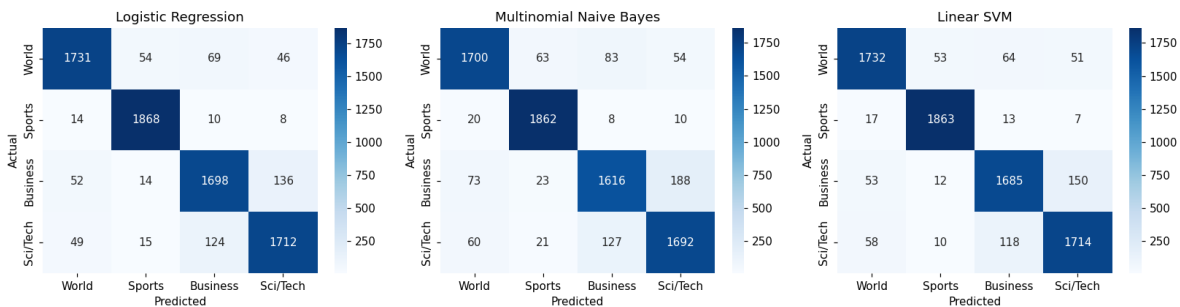


Figure 4: Confusion matrices for all three models on the test set. The dominant off-diagonal mass in every model sits between Business and Sci/Tech.

Sports is the easiest class to identify across all three models (highest recall and F1, e.g., 0.9701 F1 for Logistic Regression), reflecting its distinctive vocabulary. Business and Sci/Tech consistently show the lowest per-class scores, consistent with the confusion-matrix pattern discussed in Section 15.

14. Model Comparison

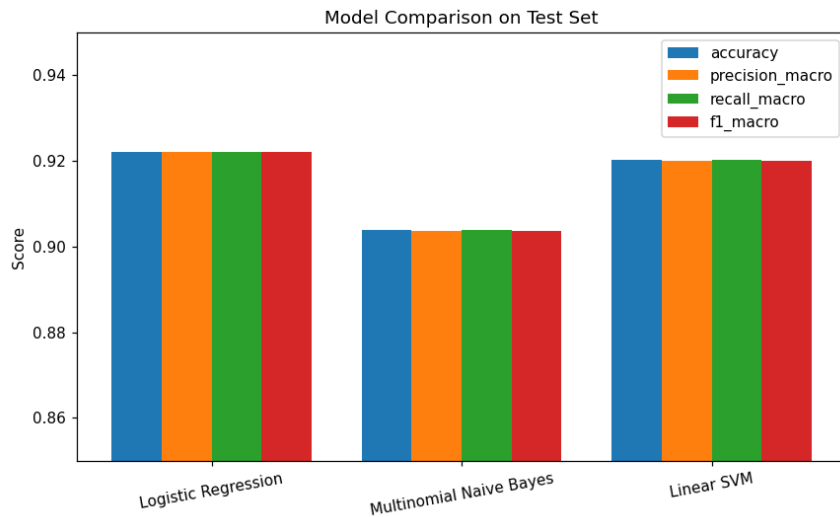


Figure 5: Side-by-side comparison of accuracy, precision, recall, and macro-F1 across the three models.

Logistic Regression and Linear SVM perform almost identically (92.22% vs. 92.03% accuracy) and both outperform Multinomial Naive Bayes by roughly 1.5–2 percentage points. This is consistent with expectations: Naive Bayes’ feature-independence assumption is a real limitation on TF-IDF features that include bigrams (which are, by construction, correlated with their component unigrams), while Logistic Regression and Linear SVM directly optimize a linear decision boundary over the full feature space and handle correlated features well.

Naive Bayes remains attractive for one dimension not captured by accuracy alone: training speed. It trained in roughly 0.03 seconds versus 4.38 seconds for Logistic Regression and 2.15 seconds for Linear SVM — a difference that would matter at much larger scale or in a resource-constrained deployment. Consistent with the objective set out in Section 5, no single model is declared “best” on accuracy alone; the trade-off between accuracy and training cost is reported explicitly.

15. Error Analysis

Misclassified test articles from the tuned Logistic Regression model were inspected directly to understand why errors occur, not just how many. The overall test error rate for this model is 7.78% (592 of 7,600 articles).

Representative Misclassified Examples

Table 8: Representative examples of misclassified test articles.

Actual Topic	Predicted Topic	Article (title + description, truncated)
Sci/Tech	Business	“GenCorp Falls 5.6 Pct. . . shares fell. . . after investment fund Steel Partners withdrew its proposal to acquire the aerospace and real estate company.”
Business	Sci/Tech	“FCC Mulls Airborne Mobile Phone Use. . . unclear why the [cell phone] ban has remained in place. . . ”
Sports	World	“Mass. court denies new trial for convict. . . father convicted of beating a man to death at their sons’ hockey practice.”
World	Sports	“Ryder Cup: Europe close on victory. . . Europe move 11-5 clear of the USA going into Sunday’s Ryder Cup singles.”
Business	Sci/Tech	“Sony, IBM, Toshiba Give Details of ‘Cell’ Chip. . . revealed their plans for the powerful new Cell processor. . . ”

Observed Error Patterns

- Overlapping topics: the most common mistake is Business $\leftrightarrow$ Sci/Tech confusion — e.g., a story about a technology company’s stock price or a corporate tech deal genuinely straddles both categories, as seen in the GenCorp and Sony/IBM/-Toshiba examples above.
- Short articles / insufficient context: with only a title and one-sentence description, some articles lack enough distinguishing vocabulary for any model to succeed.
- Ambiguous or multi-topic articles: e.g., a Sports story that is really about a legal case (a court ruling connected to a hockey-related death) can plausibly belong to World or Sports, as in the Massachusetts court example.

- Rare or named entities: company or place names appearing too infrequently in the training vocabulary (below `min_df`) provide no TF-IDF signal, forcing the model to rely on remaining generic words.
- Preprocessing limits: because n-grams only span two words, longer phrasal cues (e.g., “central bank raises interest rates”) are only partially captured.

16. Prediction Demonstration

A `predict_topic()` function wraps the exact same `clean_text()` preprocessing and the same fitted `vectorizer.transform()` step used during training, then applies the tuned Logistic Regression model. Because Logistic Regression is probabilistic, the function also reports the model’s confidence for the predicted class.

Example

Input: “Apple announced a new generation of processors for its laptop lineup, promising faster performance and better battery life.”

Output: Predicted Topic → Sci/Tech (high confidence)

Additional Examples

Table 9: Additional prediction examples from the deployed classifier.

Input Article (abridged)	Predicted Topic
“The central bank raised interest rates by half a percentage point to curb inflation.”	Business
“The home team won the championship final in overtime after a dramatic comeback.”	Sports
“Diplomats from several nations met to discuss the ongoing cease-fire negotiations.”	World
“Researchers unveiled a new quantum computing chip that could speed up drug discovery.”	Sci/Tech

All four examples were classified into their intuitively correct topic, demonstrating that the trained pipeline generalizes sensibly to freshly written text outside the AG News corpus, not just to held-out samples drawn from the same source distribution.

17. Results and Discussion

Synthesizing the evaluation, comparison, and error-analysis results (Sections 13–15): the project’s best-performing model, Logistic Regression tuned via cross-validation ($C = 1$), achieves 92.22% test accuracy and 0.9221 macro-F1 on the AG News test set, essentially matched by Linear SVM (92.03% accuracy, 0.9201 macro-F1), with Multinomial Naive Bayes trailing at 90.39% accuracy but at a small fraction of the training cost.

The results show a consistent, interpretable pattern rather than noise: Sports is classified most reliably by every model, since sports vocabulary (team names, competitions, match terminology) is highly distinctive; Business and Sci/Tech are the two hardest classes to separate, both from each other’s confusion-matrix cells and from their lower per-class precision/recall, because real news coverage of technology companies routinely blends financial and technical framing. This is a property of the underlying topic taxonomy and the source text, not a weakness specific to any one algorithm — all three models show the same qualitative confusion pattern.

These results are consistent with, and explained by, the reference paper’s own conclusions: traditional TF-IDF-based linear models are strong, competitive baselines at this dataset scale, and no single model is universally “best” — the right choice depends on whether the priority is peak accuracy (Logistic Regression / SVM) or training/deployment speed (Naive Bayes).

18. Limitations

- Short articles limit context: AG News provides only a title and a short description, not the full article body, which caps how much distinguishing information any model — traditional or deep-learning — can extract.
- Only four broad topic classes: World, Sports, Business, and Sci/Tech are coarse categories; many real articles plausibly span more than one, and finer-grained subtopics are not distinguished.
- Traditional features cannot capture deep semantics or word order beyond bigrams: TF-IDF with unigrams and bigrams misses longer phrasal structure and any deeper contextual or semantic meaning that word-order-aware or pretrained-embedding-based models can capture.
- No deep-learning or transformer comparison was trained: the project’s scope, following the reference paper’s own scale-dependent findings, restricts training to traditional TF-IDF-based classifiers; character-level ConvNets, LSTMs, and transformer models (e.g., DistilBERT) were not trained or benchmarked directly against the traditional models here.

19. Future Scope

- Extend to more topic classes: broaden the taxonomy beyond the four broad AG News categories to finer-grained subtopics (e.g., splitting Sports by sport, or Business into markets/companies/economy).
- Try word embeddings or transformer models: compare TF-IDF against pretrained word embeddings (Word2Vec/GloVe) or a fine-tuned transformer such as DistilBERT — the reference paper’s own deep-learning comparisons suggest such approaches could help further, particularly if the dataset were scaled up toward the multi-million-sample range where the paper shows character-level ConvNets pull ahead of traditional methods.
- Deploy as a live API: wrap the trained `predict_topic()` function in a lightweight web service (e.g., Flask/FastAPI) or an interactive demo (e.g., Streamlit) so the classifier can be used in real time on newly published articles.
- Use full article bodies where available, to give the model more context per document than the title-and-description snippets used here.
- Add explainability tooling (e.g., examining top TF-IDF feature weights per prediction) to make individual predictions more interpretable, extending the error-analysis work in Section 15.

20. Conclusion

This project built a complete, reproducible pipeline for automatic news topic classification on the AG News corpus: cleaning $\rightarrow$ lemmatized tokenization $\rightarrow$ TF-IDF (unigrams + bigrams) $\rightarrow$ three linear/probabilistic classifiers, evaluated with accuracy, precision, recall, and F1 (macro and weighted), supported by confusion-matrix-driven error analysis and a working single-article prediction function.

Key findings: the dataset is clean, balanced, and moderately sized (120,000 training articles across 4 classes), which fits a traditional TF-IDF plus linear-model approach well. Logistic Regression and Linear SVM achieved essentially the same, best performance ($\approx 92\%$ accuracy / macro-F1), meaningfully ahead of Multinomial Naive Bayes ($\approx 90\%$). The dominant error mode is Business $\leftrightarrow$ Sci/Tech confusion, reflecting genuine topical overlap in real news rather than a flaw specific to any one model. Hyperparameter tuning confirmed, rather than assumed, that the default regularization strength was already close to optimal for this feature space.

The project satisfies its stated objectives: a well-defined problem, a properly sourced and described dataset, documented preprocessing, appropriate feature extraction, multiple compared models, honest evaluation, error analysis, a working prediction system, and an explicit, evidence-based connection to the reference literature.

21. References

1. Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. arXiv:1509.01626. <https://arxiv.org/abs/1509.01626>
2. Jones, K. S. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. Documentation: <https://scikit-learn.org/stable/documentation.html>
4. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O’Reilly Media. NLTK Documentation: <https://www.nltk.org/>
5. Gulli, A. AG’s Corpus of News Articles. http://www.di.unipi.it/~gulli/AG_corpus_of_news_articles.html (original dataset source referenced in Zhang, Zhao & LeCun, 2015, Section 4).
6. amananandrai. AG News Classification Dataset. Kaggle. <https://www.kaggle.com/datasets/amananandrai/ag-news-classification-dataset> (the specific `train.csv` / `test.csv` files used in this project).

End of Report